# Simple Linear Regression: The Mathematics from Scratch

## Contents


1. Overview . . . 2
2. Two ways to find m and b . . . 2
3. The error function . . . 4
   - 3.1 The error at one point . . . 4
   - 3.2 Adding the errors up . . . 6
   - 3.3 E depends on m and b . . . 8
4. Finding the minimum . . . 9
   - 4.1 The shape of E . . . 9
   - 4.2 A hand run on the three toy points . . . 11
   - 4.3 Step 1: the derivative with respect to b . . . 13
   - 4.4 Step 2: the derivative with respect to m . . . 14
   - 4.5 The two formulas . . . 15
5. The formulas on the placement data . . . 17
   - 5.1 Another way to see the slope: correlation . . . 18
6. Our own linear regression class . . . 19
7. Summary . . . 22
8. Sources . . . 22
9. Key terms . . . 23

**Where this fits:** Pipeline map step 8 (Model). See the Course map.

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 Foundations | 1 Frame | 2 Get data | 3 Understand | 4 Clean | 5 Features | 6 Reduce |
| 7 Split | **8 Model** | 9 Evaluate | 10 Tune | 11 Deploy | 12 Test | 13 Monitor |

- **Builds on:** Covariance and covariance matrix; Simple linear regression.
- **Leads to:** Normal equation.
- **Compare with:** Gradient descent.

## 1. Overview

**Key point:** We write down the total error of a line, find the m and b that make it smallest, and get two short formulas. Coded as our own class, they give exactly scikit-learn's answer.

The previous Note trained `LinearRegression` and read off the slope $m = 0.558$ and intercept $b = -0.896$ for the placement data. This Note opens the box: where do these two numbers come from? As before, the **feature** (G-772; an input variable, one column of the data table) is CGPA, the **target** (G-1949; the output we predict) is the package, and each **observation** (G-1374; one record, one row) is one student. The line itself is the **best-fit line** (G-280) of **linear regression** (G-1094).

The plan has three parts:

1. Write the total error of any line as a formula in $m$ and $b$ (Section 3).
2. Find the $m$ and $b$ that make it smallest, using **derivatives** (G-595) (Section 4).
3. Code the result as our own class and compare it with scikit-learn (Section 6).

## 2. Two ways to find m and b

**Key point:** A closed-form solution gives m and b directly from a formula (OLS). A non-closed-form solution reaches them step by step (gradient descent).

There are two ways to compute the best $m$ and $b$: jump straight to the answer, or walk towards it.

- **Closed-form solution** (G-398): a formula we can evaluate directly, using only ordinary operations such as adding, multiplying and dividing. The quadratic formula from school is an example: it gives the answer in one go. For **linear regression** (G-1094), this method is called **ordinary least squares (OLS)** (G-1406).
- **Non-closed-form solution** (G-1332): no direct formula; we start from a guess and improve it step by step until it is good enough. For linear regression, this method is **gradient descent** (G-862).

Both routes search for the same thing, so first we need the picture of what is searched. Every line is fixed by two numbers, the slope $m$ and the intercept $b$, and every line has a **total error** $E(m, b)$: the sum of the squared gaps between the line and the 160 training students (defined

step by step in Section 3). For example, the flat line $m = 0$, $b = 0$ has $E(0, 0) = 1{,}516.8$, and the best line, $m = 0.558$, $b = -0.896$, has $E = 16.55$.

Take $(m, b)$ as a point on the floor and $E$ as the height above it. Then $E$ is a surface. It is a long, narrow valley (a bowl stretched in one direction) with one lowest point, the best line. Figure 1 draws the surface with its height as $\log_{10} E$ (the power of 10 that gives $E$; $\log_{10} 100 = 2$), so that the steep walls do not hide the floor. Thin lines on the surface join points at the same height; they are also dropped onto the floor. The camera then tilts until it looks straight down. A contour map is exactly this view from above: each line joins points of the same height. Lines close together mean a steep wall; the centre of the dark band is the lowest point, the black cross.

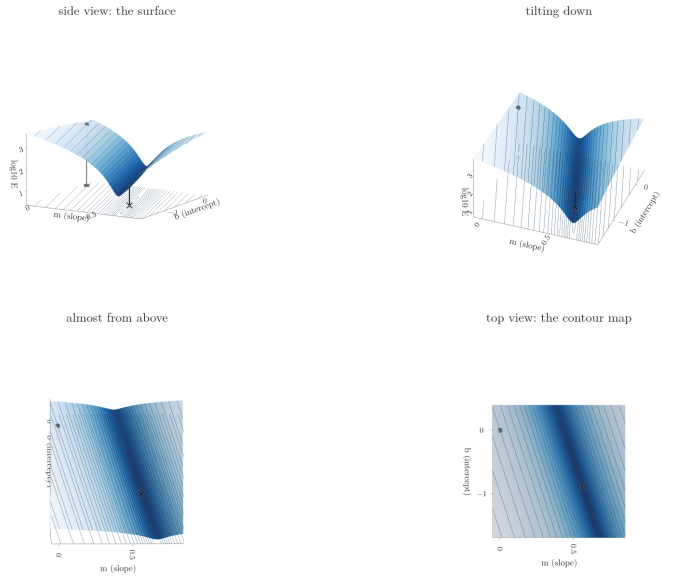



**Figure 1:** The error surface E(m, b) on the 160 training students, seen from the side and then tilted to the top view. Height is log10 E. Black cross: the best line ($m = 0.558$, $b = -0.896$). Grey square: the start m = 0, b = 0. The final frame is the contour map used in Figure 2 and Figure 6.

Figure 2 runs both on the placement data. Each point of the map on the right (the final frame of Figure 1) is one line $(m, b)$, coloured by its total error (Section 3); the black cross is the best line. OLS lands on the cross in one jump. Gradient descent starts at $m = 0$, $b = 0$ and takes small steps downhill: it first swings $m$ past the answer, then crawls along the long, narrow valley, and needs 6,112 steps to get within 0.001 of the OLS slope and 0.01 of the OLS intercept.

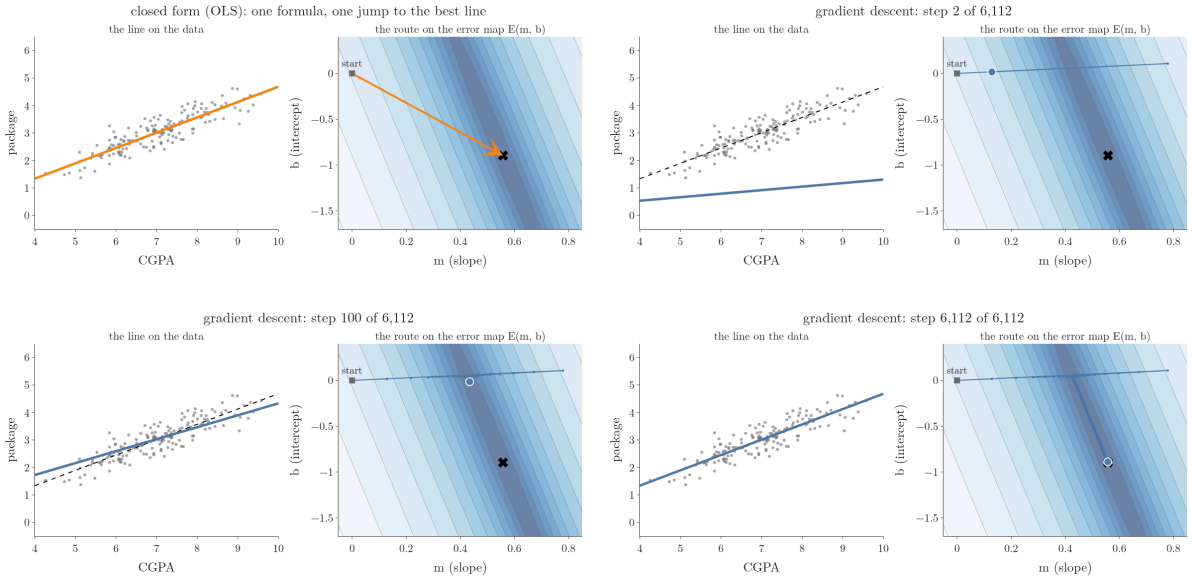



**Figure 2:** Two routes to the best line on the 160 training students. Orange: OLS computes $m = 0.558$, $b = -0.896$ in one step. Blue: gradient descent from (0, 0), learning rate 0.018, reaches the same line after 6,112 small steps. Left: the line on the data; right: the route on the error map.

Why have both? With one or a few features, OLS is fast and exact. With very many features, the OLS formula becomes expensive to compute, because it needs the inverse of a matrix with one row and one column per feature (the cost of the inverse), and gradient descent works better.

scikit-learn uses both: `LinearRegression` uses OLS, and `SGDRegressor` uses gradient descent. This Note derives OLS; **gradient descent** (G-862) is taught separately.

## 3. The error function

> **Key point:** The error of a line is the sum of the squared vertical gaps between each point and the line. The error depends only on m and b.

### 3.1 The error at one point

> **Key point:** For each point, the error is the actual value minus the value the line predicts.

The **best-fit line** (G-280) should pass as close as possible to all the points. For each point, the vertical gap between the point and the line measures how wrong the line is there (Figure 3).

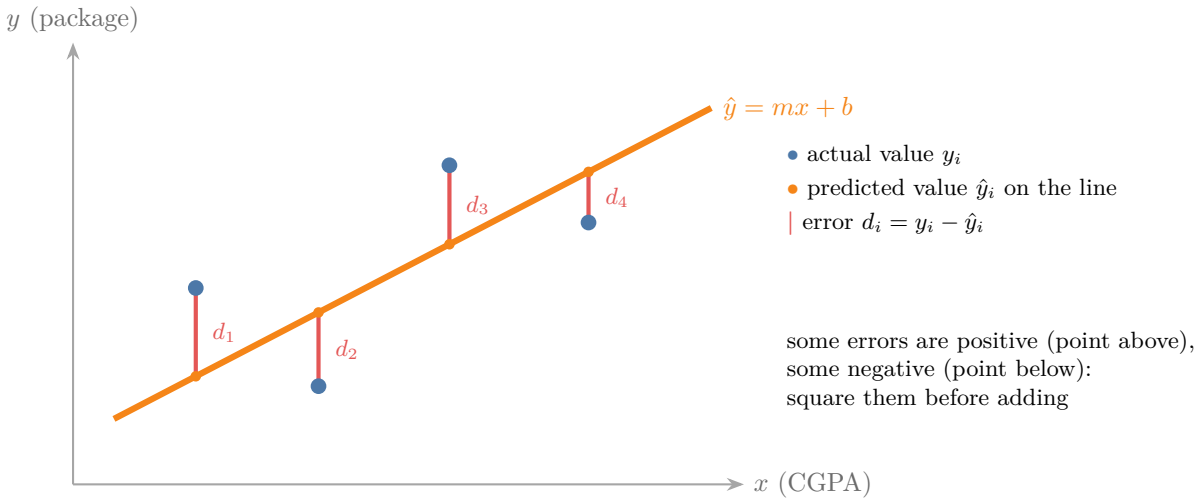



**Figure 3:** The error at each point is the vertical gap to the line

For point $i$, the actual package is $y_i$ and the line predicts $\hat{y}_i$, the **prediction** (G-1550; read "y-hat", a hat marks a prediction). The error at the point is

$$d_i = y_i - \hat{y}_i$$

This signed vertical gap is the **residual** (G-705) of observation $i$: positive when the point lies above the line, negative when it lies below.

**A three-point toy set.** To see every step by hand, we use three students before the 160. The index $i$ counts the observations (1, 2, 3), $x_i$ is the CGPA and $y_i$ the package:

| $i$ | $x_i$ (CGPA) | $y_i$ (package) |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 3 |
| 3 | 3 | 2 |

Try the guess line $\hat{y} = x$, that is slope 1 and intercept 0. Its predictions and residuals:

| $i$ | $\hat{y}_i = x_i$ | $d_i = y_i - \hat{y}_i$ |
|---|---|---|
| 1 | 1 | $1 - 1 = 0$ |
| 2 | 2 | $3 - 2 = 1$ |
| 3 | 3 | $2 - 3 = -1$ |

### 3.2 Adding the errors up

> **Key point:** Squaring makes every error positive and punishes large errors more; it also keeps the function smooth enough to differentiate.

To get the total error we add the errors of all $n$ points. Adding $d_1 + d_2 + \cdots + d_n$ directly does not work: points above the line give positive errors, points below give negative ones, and they cancel out.

Figure 4 shows the cancelling on our data. A flat line at the average package is clearly a poor fit, yet its **residuals** (G-705) add up to exactly 0, the same as the best line's. The raw sum cannot tell the two lines apart; the sum of the squares can (73.02 against 16.55).

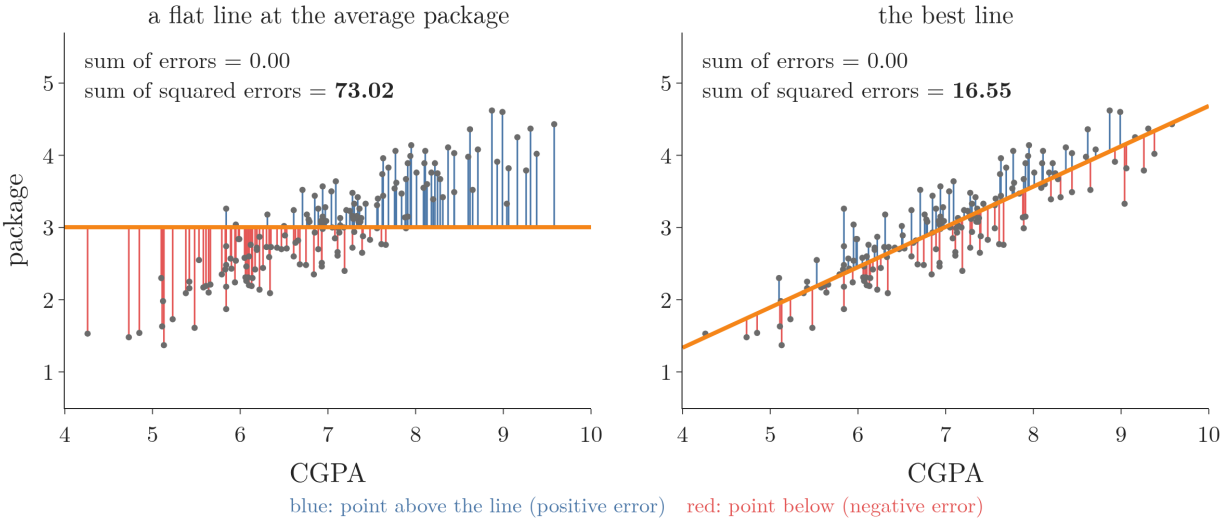



**Figure 4:** Two lines on the 160 training students. Blue sticks: positive residuals; red sticks: negative ones. For both lines the residuals add up to 0, but the sums of squared residuals are 73.02 for the flat line and 16.55 for the best line.

Adding the toy residuals shows the cancelling:

$$0 + 1 + (-1) = 0$$

The line misses two points, yet its total is 0. Squaring each residual first:

$$0^2 + 1^2 + (-1)^2 = 0 + 1 + 1 = 2$$

So we square each error before adding. Squares were chosen over absolute values $|d_i|$ for two reasons:

- **Large errors count more:** an error of 2 counts 4, an error of 10 counts 100. A line that badly misses some points is punished.
- **The square can be differentiated:** in Section 4 we find the minimum with **derivatives** (G-595). The absolute value has a sharp corner at 0, where it has no derivative; the square is smooth everywhere.

Figure 5 puts the two choices side by side: the orange square grows faster for large residuals, and it has no corner at 0.

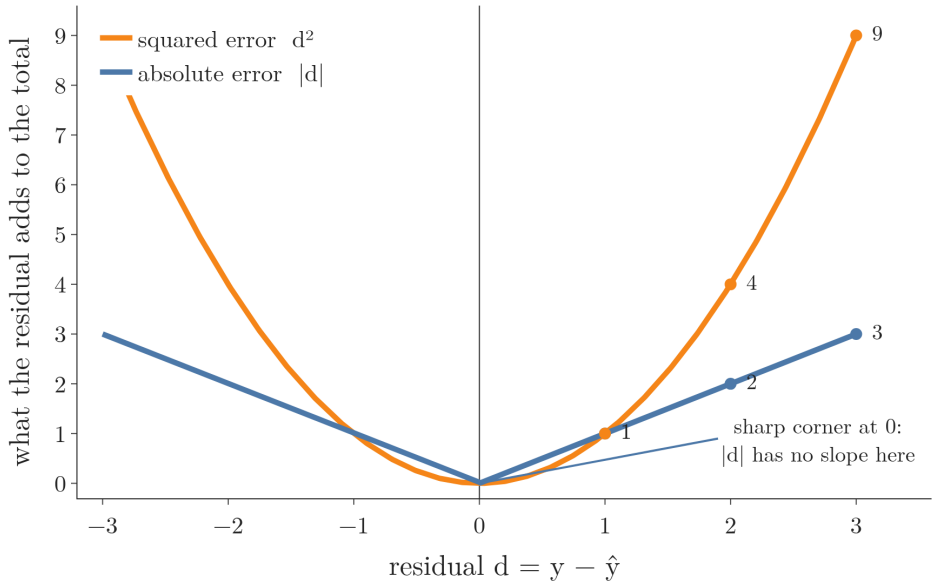



**Figure 5:** What one residual d adds to the total: d squared (orange) against the absolute value of d (blue). Residuals of 1, 2 and 3 add 1, 4 and 9 when squared.

The total is called the **error function** or **loss function** (G-706); for squared errors it is the **sum of squared errors** (G-1684). The symbol $\sum_{i=1}^{n}$ (sigma) means "add the term for $i = 1$, then $i = 2$, and so on up to $i = n$"; with $n = 3$ toy points the guess line gives $E = 2$, the sum we just wrote. In general:

$$E = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

> **Extra:** Dividing $E$ by $n$ gives the average squared error, the **mean squared error** (G-1201) used as a metric. Dividing by a constant does not change which line is best, so here we keep the plain sum.

### 3.3 E depends on m and b

> **Key point:** The data is fixed; only m and b can change. So the error is a function of m and b.

Every prediction comes from the line: $\hat{y}_i = mx_i + b$. Putting this into $E$:

$$E(m, b) = \sum_{i=1}^{n} (y_i - mx_i - b)^2$$

The values $x_i$ (CGPA) and $y_i$ (package) are the data: we cannot change them. Only $m$ and $b$ can change. Turning a line in the plane means changing its slope $m$; sliding it up or down means changing its intercept $b$. Every possible line is one pair $(m, b)$, and each pair has its own total error.

On the toy set, $E(m, b)$ is three terms, one per student:

$$E(m, b) = (1 - m - b)^2$$

$$+ (3 - 2m - b)^2$$

$$+ (2 - 3m - b)^2$$

The guess line has $m = 1$, $b = 0$:

$$E(1, 0) = (1 - 1 - 0)^2 + (3 - 2 - 0)^2$$

$$+ (2 - 3 - 0)^2$$

$$= 0 + 1 + 1$$

$$= 2$$

So the task becomes: **find the $m$ and $b$ that make $E(m, b)$ as small as possible.**

Figure 6 links the two views. On the left, each error is drawn as a square with side $|d_i|$; both axes use the same unit, so $E$ is the total orange area. On the right, the same line is a single dot on a map of $E(m, b)$, where darker means smaller. First the line turns with $b$ fixed, then it slides with $m$ fixed. Watch the squares shrink and grow: each time the area is smallest when the dot crosses the black cross, the line with $m = 0.558$ and $b = -0.896$.

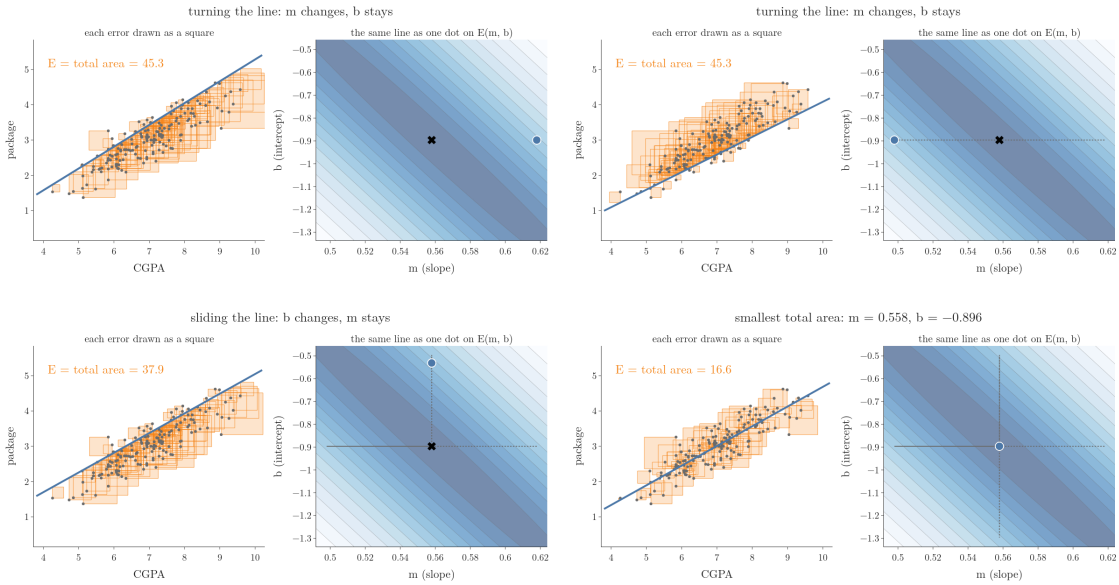



**Figure 6:** Turning and sliding a line through the 160 training students. Left: the squared errors as squares, with total area E. Right: the same line as a dot on the contour map of E(m, b); the black cross marks the smallest E.

## 4. Finding the minimum

> **Key point:** At the lowest point of E, its slope is zero in the m direction and in the b direction. Setting both **derivatives** (G-595) to zero gives two equations for m and b.

### 4.1 The shape of E

> **Key point:** E is a bowl. Its lowest point is the best line, and there the bowl is flat in every direction.

Figure 1 already showed this surface (as $\log_{10} E$) and flattened it into the contour map. Figure 7 zooms in on the bottom and draws $E(m, b)$ itself, not its logarithm: each point of the surface is one line, and its height is that line's total error.

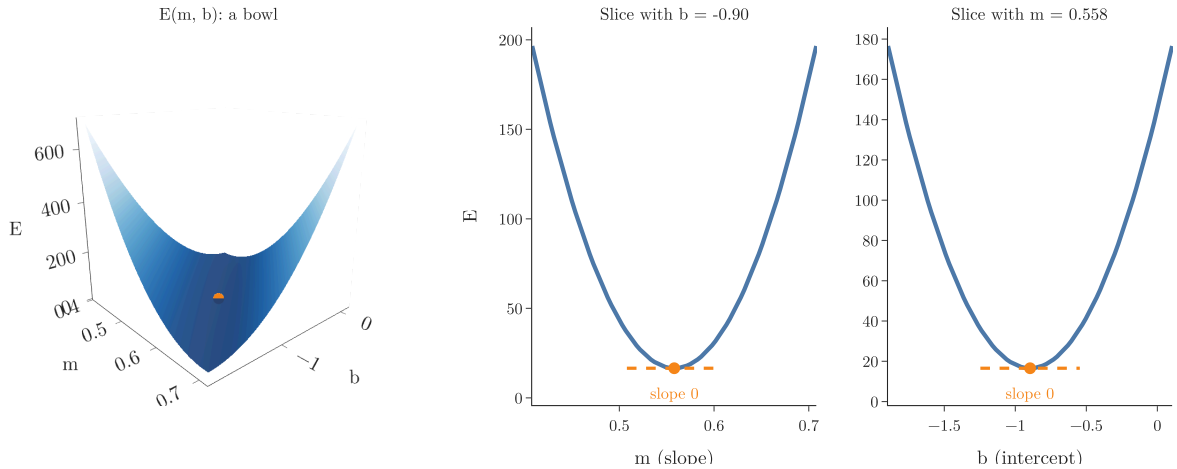



**Figure 7:** The error function: a bowl with the best line at the bottom

The surface is a bowl with a single lowest point, at $m = 0.558$, $b = -0.896$, where $E = 16.55$. The two slices on the right cut the bowl along $m$ and along $b$: each is a U-shaped curve, and at the bottom the curve is flat; its slope there is zero. A marble dropped into a salad bowl comes to rest at the one place where the floor of the bowl is level in every direction.

A **derivative** (G-595) measures the slope of a function: the slope of its **tangent line** (G-1945), the straight line that touches the curve at one point. Since $E$ depends on two variables, it has two slopes, one for each. A **partial derivative** (G-1457) is the slope in one variable while the other is held fixed:

- $\dfrac{\partial E}{\partial b}$: how $E$ changes when $b$ moves and $m$ stays fixed.
- $\dfrac{\partial E}{\partial m}$: how $E$ changes when $m$ moves and $b$ stays fixed.

Figure 8 slides a tangent line along each slice of Figure 7. With $m = 0.558$ fixed, at $b = -1.9$ the tangent slopes down, $\partial E/\partial b = -321$: raising $b$ lowers the error. At $b = 0.1$ it slopes up, $\partial E/\partial b = 319$. Only at $b = -0.896$ is the tangent flat. The $m$ slice behaves the same way ($-1{,}727$ at $m = 0.45$, $1{,}473$ at $m = 0.65$). At the minimum, both partial derivatives are zero.

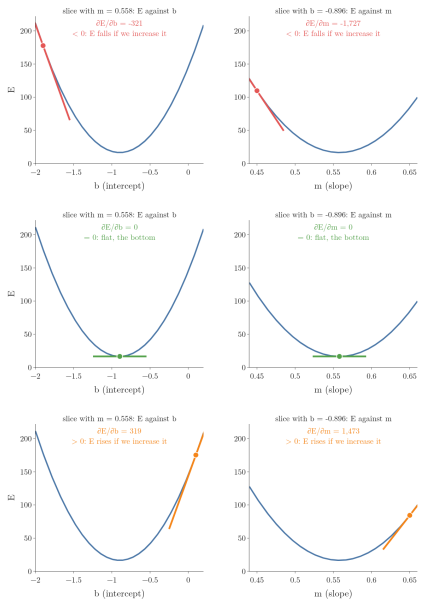



**Figure 8:** A tangent line slides along each slice of the bowl. Its slope is the partial derivative: negative (red) before the bottom, 0 (green) at $m = 0.558$ and $b = -0.896$, positive (orange) after.

### 4.2 A hand run on the three toy points

> **Key point:** Setting both slopes of the toy $E(m, b)$ to zero gives two small equations; solving them gives the best line, $m = 0.5$ and $b = 1$.

First the slope of a single squared term, using the **chain rule** (G-371): when one quantity sits inside another, the slope of the whole is the outer slope times the inner slope. Take student 2 on the line $m = 0.5$, $b = 1$:

$$u = y - mx - b$$

$$u = 3 - 0.5 \times 2 - 1 = 1$$

$$u^2 = 1$$

Raise $b$ by 0.1. Then $u$ falls by 0.1, from 1 to 0.9:

$$u^2 = 0.9^2 = 0.81$$

$$\text{change} = 0.81 - 1 = -0.19$$

The chain rule predicts the same change without trying it. The outer slope of $u^2$ is $2u$, which is 2 here. The inner slope of $u$ with respect to $b$ is $-1$. Multiplying them gives the slope:

$$2 \times (-1) = -2$$

For a step of 0.1 in $b$:

$$-2 \times 0.1 = -0.2 \approx -0.19$$

Now the whole toy error. Each of the three terms has inner slope $-1$ for $b$ and $-x_i$ for $m$ (raising $m$ by 1 lowers $u$ by $x_i$):

$$\frac{\partial E}{\partial b} = -2\,[\,(1 - m - b)$$

$$+\,(3 - 2m - b)$$

$$+\,(2 - 3m - b)\,]$$

$$= -2\,(6 - 6m - 3b)$$

$$\frac{\partial E}{\partial m} = -2\,[\,1(1 - m - b)$$

$$+\,2(3 - 2m - b)$$

$$+\,3(2 - 3m - b)\,]$$

$$= -2\,(13 - 14m - 6b)$$

Set both to zero, drop the factor $-2$, and solve:

$$6 - 6m - 3b = 0$$

$$b = 2 - 2m$$

$$13 - 14m - 6(2 - 2m) = 0$$

$$1 - 2m = 0$$

$$m = 0.5$$

$$b = 2 - 2 \times 0.5 = 1$$

Check: the line $\hat{y} = 0.5x + 1$ predicts 1.5, 2, 2.5; the **residuals** (G-705) are $-0.5$, 1, $-0.5$.

$$E = 0.25 + 1 + 0.25 = 1.5$$

This is below the guess line's 2. Both conditions hold. The residuals add up to 0:

$$-0.5 + 1 - 0.5 = 0$$

The residuals times $x_i$ add up to 0:

$$-0.5 + 2 - 1.5 = 0$$

Sections 4.3 and 4.4 repeat these same steps with $n$ points and letters.

### 4.3 Step 1: the derivative with respect to b

> **Key point:** Setting the b-derivative to zero gives $b = \bar{y} - m\bar{x}$.

In words: differentiate each squared term with respect to $b$, set the sum to zero, and solve for $b$. The **chain rule** (G-371) worked on the toy points in Section 4.2 gives each term the slope "2 times the gap, times $-1$":

$$\frac{\partial E}{\partial b} = \sum_{i=1}^{n} 2(y_i - mx_i - b)(-1) = 0$$

Divide both sides by $-2$ and split the sum into three parts:

$$\sum y_i - m \sum x_i - nb = 0$$

Divide by $n$. The **mean** (G-1203, the average) appears:

$$\bar{x} = \frac{1}{n} \sum x_i$$

and likewise $\bar{y}$:

$$\bar{y} - m\bar{x} - b = 0$$

$$b = \bar{y} - m\bar{x}$$

The formula for $b$ says the best line always passes through the point $(\bar{x}, \bar{y})$: the average CGPA and the average package. Once we know $m$, $b$ follows.

### 4.4 Step 2: the derivative with respect to m

> **Key point:** Setting the m-derivative to zero and using $b = \bar{y} - m\bar{x}$ gives the formula for m.

In words: differentiate with respect to $m$, set it to zero, replace $b$ with the result of Step 1, and solve for $m$. Now the **chain rule** (G-371) gives each term "2 times the gap, times $-x_i$", as in Section 4.2.

$$\frac{\partial E}{\partial m} = \sum_{i=1}^{n} 2(y_i - mx_i - b)(-x_i) = 0$$

Replace $b$ by $\bar{y} - m\bar{x}$ and divide by $-2$:

$$\sum_{i=1}^{n} \left[(y_i - \bar{y}) - m(x_i - \bar{x})\right] x_i = 0$$

Split the sum into two:

$$\sum (y_i - \bar{y})\, x_i - m \sum (x_i - \bar{x})\, x_i = 0$$

The deviations from a mean add up to zero. On the toy points, $x_i - \bar{x} = -1, 0, 1$ add to 0, and so do $y_i - \bar{y}$. So subtracting $\bar{x} \sum (y_i - \bar{y})$ from the first sum changes nothing, because it is

$$\bar{x} \times 0 = 0$$

and it turns $x_i$ into $x_i - \bar{x}$:

$$\sum (y_i - \bar{y})\, x_i = \sum (y_i - \bar{y})(x_i - \bar{x})$$

The same step on the second sum:

$$\sum (x_i - \bar{x})\, x_i = \sum (x_i - \bar{x})^2$$

The equation becomes

$$\sum(y_i - \bar{y})(x_i - \bar{x}) - m\sum(x_i - \bar{x})^2 = 0$$

Moving the second term across and dividing gives the formula for $m$:

$$m = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

> **Extra:** The last step in more detail. The equation says $\sum(y_i - \bar{y})x_i = m\sum(x_i - \bar{x})x_i$. Since $\sum(y_i - \bar{y}) = 0$, we may subtract $\bar{x}\sum(y_i - \bar{y})$ from the left side without changing it, which turns $x_i$ into $(x_i - \bar{x})$. The same trick works on the right side, since $\sum(x_i - \bar{x}) = 0$. Dividing by the right-hand sum gives the formula.

### 4.5 The two formulas

> **Key point:** m is the covariance of x and y divided by the variance of x; b makes the line pass through the means.

| | Formula |
|---|---|
| Slope | $m = \dfrac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$ |
| Intercept | $b = \bar{y} - m\bar{x}$ |

On the toy points, $\bar{x} = 2$ and $\bar{y} = 2$. The top of the formula is a sum of three products:

$$(1 - 2)(1 - 2) = 1$$

$$(2 - 2)(3 - 2) = 0$$

$$(3 - 2)(2 - 2) = 0$$

$$1 + 0 + 0 = 1$$

The bottom:

$$(1 - 2)^2 + (2 - 2)^2 + (3 - 2)^2$$

$$= 1 + 0 + 1 = 2$$

$$m = \frac{1}{2} = 0.5$$

$$b = 2 - 0.5 \times 2 = 1$$

These are the values found by hand in Section 4.2. Now in words: the top of the $m$ formula is $n$ times the **covariance** (G-496) of $x$ and $y$; the bottom is $n$ times the **variance** (G-2074) of $x$ (see covariance and **variance** (G-2074)). So the slope is how much $x$ and $y$ move together, divided by how much $x$ moves on its own.

Figure 9 shows what the top of the formula adds up. Move the axes to the point of means. A student with both CGPA and package above average (top right) gives a positive product $(x_i - \bar{x})(y_i - \bar{y})$; so does a student below average on both (bottom left). A student above on one and below on the other gives a negative product. In the placement data 125 of the 160 products are positive and add up to 103.06; the 35 negative ones add up to only $-1.86$. So the sum is 101.204, a clearly upward slope.

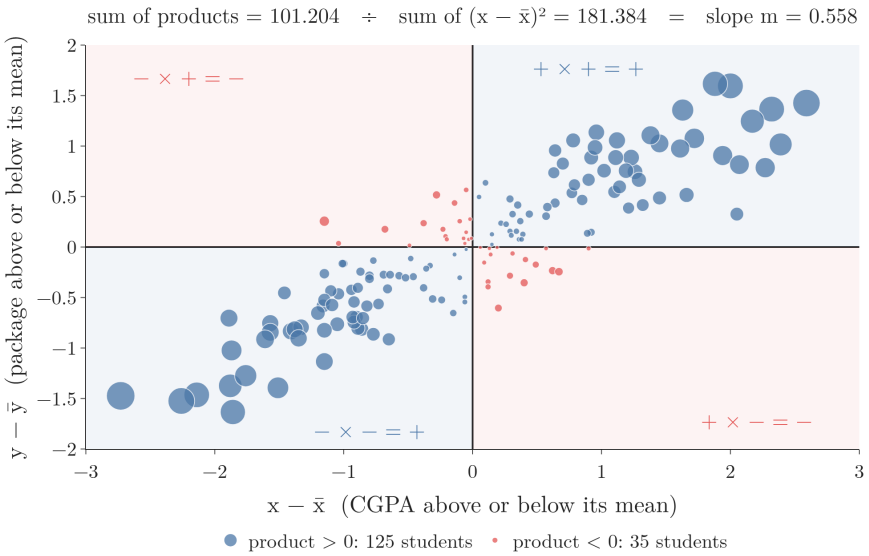



**Figure 9:** The 160 training students with the axes moved to the point of means. Each dot's size is the size of its product; blue products are positive, red negative. Sum of products 101.204, divided by the sum of squared CGPA deviations 181.384, gives m = 0.558.

> **Extra:** Strictly, a zero slope only shows a flat point, which could be a maximum or a saddle. For $E(m, b)$ the flat point is always a minimum: $E$ is a sum of squares of linear expressions, a bowl that only curves upward (Figure 7). The check uses the second

**derivatives** (G-595):

$$\frac{\partial^2 E}{\partial b^2} = 2n$$

$$\frac{\partial^2 E}{\partial m^2} = 2\sum x_i^2$$

$$\frac{\partial^2 E}{\partial m\,\partial b} = 2\sum x_i$$

The first is positive. The test also needs this combination to be positive:

$$\frac{\partial^2 E}{\partial b^2}\cdot\frac{\partial^2 E}{\partial m^2} - \left(\frac{\partial^2 E}{\partial m\,\partial b}\right)^2$$

$$= 4\left(n\sum x_i^2 - (\sum x_i)^2\right)$$

$$= 4n\sum(x_i - \bar{x})^2$$

The last line is positive whenever the $x_i$ are not all equal. By the second-derivative test for two variables, a flat point with both of these positive is a minimum.

## 5. The formulas on the placement data

**Key point:** On the 160 training students, the formulas give m = 0.558 and b = −0.896, exactly the values scikit-learn found.

The sums over 160 students are added up by the computer (the code in Section 6: input `X_train` and `y_train`, output the two sums below), using the same lines as the three-point check in Section 4.5. With the training students of the previous Note (same 80/20 split):

1. **Means:** $\bar{x} = 6.9899$ (average CGPA), $\bar{y} = 3.0039$ (average package).
2. **Top of the m formula:** $\sum(x_i - \bar{x})(y_i - \bar{y}) = 101.204$.
3. **Bottom of the m formula:** $\sum(x_i - \bar{x})^2 = 181.384$.

Then

$$m = \frac{101.204}{181.384} = 0.55795 \approx 0.558$$

$$b = 3.0039 - 0.55795 \times 6.9899$$

$$b = 3.0039 - 3.9000 = -0.896$$

Figure 10 draws this line on the training students. As Step 1 promised, the line passes exactly through the point of means $(\bar{x}, \bar{y})$.

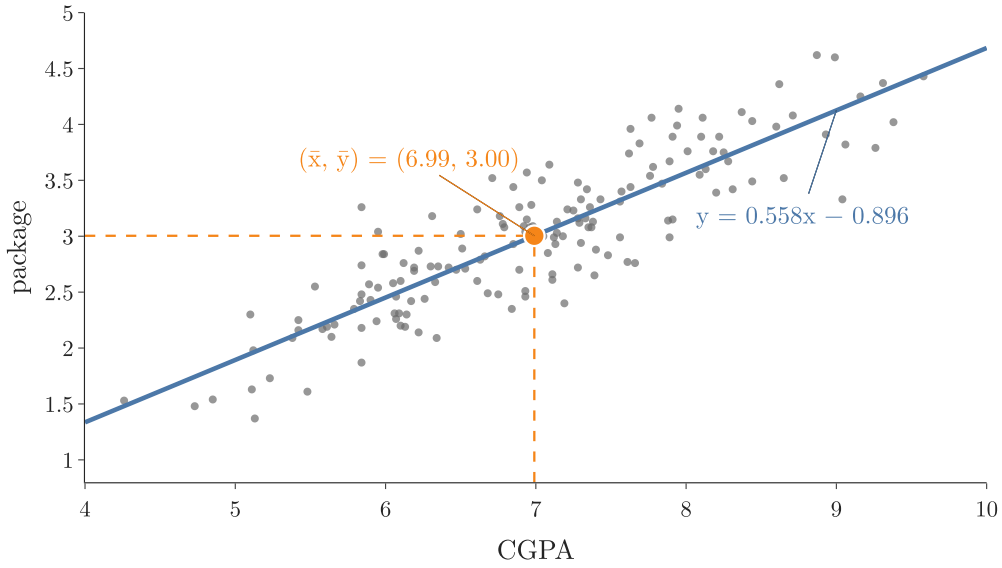



**Figure 10:** The OLS line on the 160 training students passes through the point of means (6.99, 3.00)

These are the numbers `LinearRegression` reported. scikit-learn reaches them by a different route: instead of these sums, `LinearRegression` solves the same least-squares problem with a matrix method, `scipy.linalg.lstsq` (scikit-learn docs, LinearRegression). The same problem has the same answer, so both give the same $m$ and $b$.

### 5.1 Another way to see the slope: correlation

> **Key point:** The slope is the correlation times the ratio of the two spreads: $m = r \times s_y / s_x$.

The $m$ formula can be rewritten with the **Pearson correlation coefficient** $r$ (G-1474) (correlation). Write $s_x$ and $s_y$ for the standard deviations of CGPA and package. Correlation is **covariance** (G-496) divided by both standard deviations, so covariance $= r \times s_x \times s_y$, and

$$m = \frac{\text{covariance}}{\text{variance of } x}$$

$$m = \frac{r \times s_x \times s_y}{s_x^2}$$

$$m = r \times \frac{s_y}{s_x}$$

On the training students $r = 0.879$, $s_x = 1.068$ and $s_y = 0.678$:

$$m = 0.879 \times \frac{0.678}{1.068} = 0.558$$

Figure 11 reads this as a recipe for drawing the line:

1. Start at the point of means $(\bar{x}, \bar{y})$; the line always passes through it (Section 4.3).
2. Step right by one standard deviation of CGPA, $s_x = 1.07$.
3. Step up by $r \times s_y$ (the correlation times the spread of the package):

$$0.879 \times 0.678 = 0.60$$

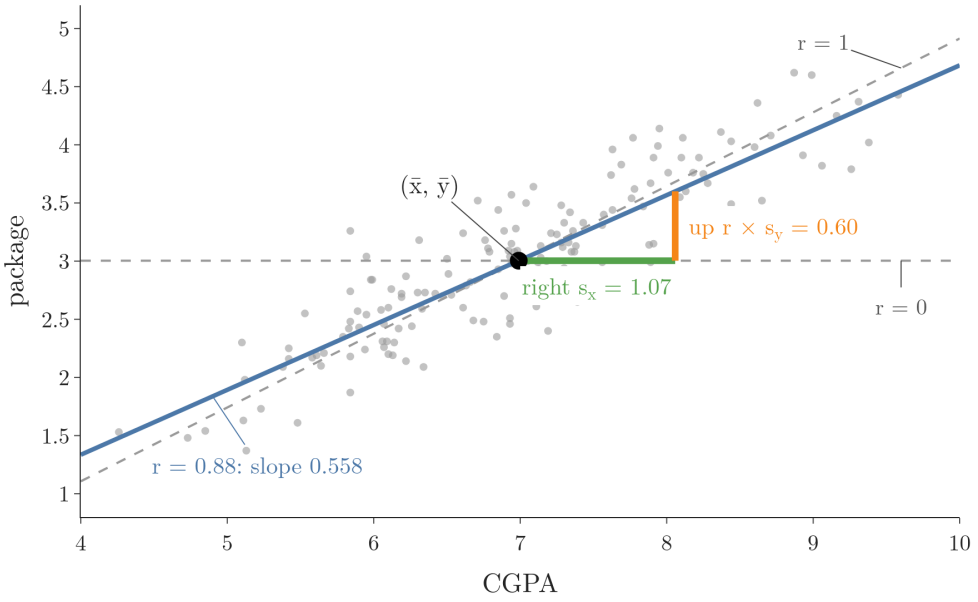



**Figure 11:** The best-fit line drawn from the point of means: right by $s_x$, up by $r \times s_y$. Dashed: the line if r were 1 (up a full s_y) and if r were 0 (flat at the average package). Idea after Khan Academy, "Calculating the equation of a regression line".

The dashed lines in Figure 11 are the two extremes. With a perfect correlation, $r = 1$, the line would rise a full $s_y$ for each $s_x$. With no correlation, $r = 0$, the line would be flat at the average package: CGPA would tell us nothing, and the best prediction would be the average for everyone. Our $r = 0.879$ takes the line 88 percent of the way from flat to the $r = 1$ line.

## 6. Our own linear regression class

> **Key point:** A class with fit (the two formulas) and predict (the line equation) reproduces scikit-learn's LinearRegression for one feature.

**Python:** Simple linear regression from scratch.

```
class MyLR:
    def fit(self, X, y):
        # closed-form OLS formulas from Section 4.5
        x_bar, y_bar = X.mean(), y.mean()
        self.m = (((X - x_bar) * (y - y_bar)).sum()
                  / ((X - x_bar) ** 2).sum())
        self.b = y_bar - self.m * x_bar
        return self

    def predict(self, X):
        return self.m * X + self.b      # the line

mine = MyLR().fit(X_train, y_train)
mine.m, mine.b                  # 0.558, -0.896
mine.predict(X_test[:3])        # [3.8911, 3.0932, 2.3846]
```

`X_train` here is a plain 1-D NumPy array of CGPAs. NumPy works on whole arrays at once, so the sums need no loop.

The predictions match scikit-learn's to every digit shown. Figure 12 draws the class's output on the 40 test students: our line lies exactly on scikit-learn's.

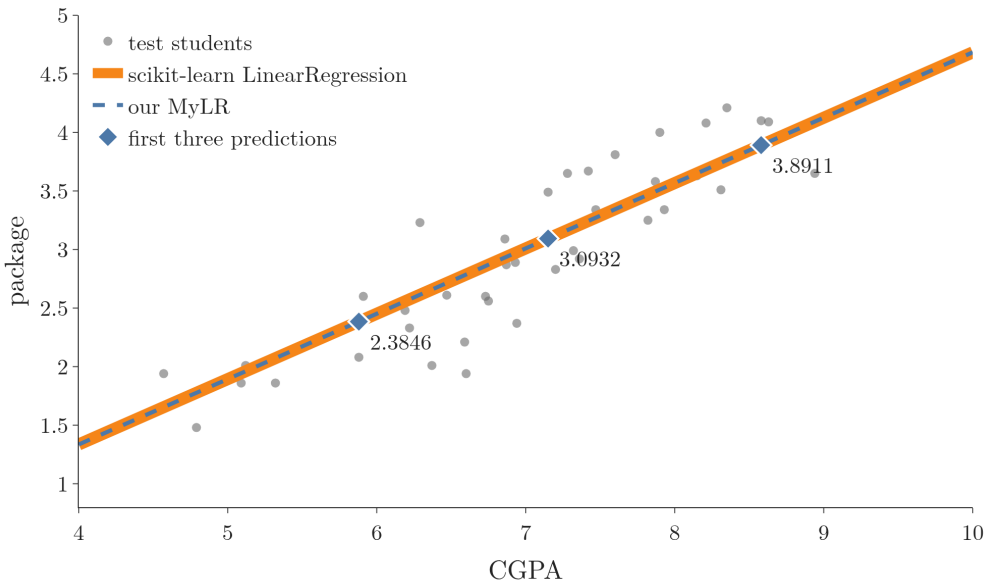



**Figure 12:** On the 40 test students, the line of our MyLR class (dashed blue) lies on top of scikit-learn's LinearRegression line (orange). Diamonds: the first three test predictions, 3.8911, 3.0932 and 2.3846.

The Notebook also checks the two conditions of Section 4 at the fitted line: the errors add up to 0, and so do the errors times $x$. Moving $m$ by just 0.01 raises $E$ from 16.55 to 17.35.

**Extra:** This class only handles one feature. The same idea works for many features, written with matrices instead of single sums: **multiple linear regression** (G-1279).

## 7. Summary

| Step | Result | Why it matters |
|---|---|---|
| Error at a point | $d_i = y_i - \hat{y}_i$ | it measures how wrong the line is at each student |
| Error function | $E(m, b) = \sum(y_i - mx_i - b)^2$ | the data is fixed, so finding the best line means finding the best $m$ and $b$ |
| Minimum | $\partial E/\partial m = 0$ and $\partial E/\partial b = 0$ | the bowl is flat in both directions only at its bottom |
| Intercept | $b = \bar{y} - m\bar{x}$ | once $m$ is known, $b$ follows |
| Slope | $m = \sum(x_i - \bar{x})(y_i - \bar{y}) \,/\, \sum(x_i - \bar{x})^2$ | how much $x$ and $y$ move together, divided by how much $x$ moves alone |
| Placement data | $m = 0.558$, $b = -0.896$, $E = 16.55$ | the same numbers `LinearRegression` reported |

- Closed form (OLS) gives $m$ and $b$ directly; **gradient descent** (G-862) reaches them step by step, so OLS suits one or a few features and gradient descent suits very many, where the OLS formula gets expensive.
- Errors are squared so they do not cancel, large errors count more, and the function can be differentiated.
- The error function is a bowl in $(m, b)$; the best line is its bottom, where both **partial derivatives** (G-1457) are zero, so setting the two derivatives to zero gives two equations that pin down $m$ and $b$.
- The best line always passes through $(\bar{x}, \bar{y})$, because the $b$-derivative set to zero gives exactly that condition.
- The slope is also $r \times s_y/s_x$: correlation times the ratio of the spreads, so with $r = 0$ the line is flat at the average and CGPA tells us nothing.
- scikit-learn's `LinearRegression` solves the same least-squares problem (with `scipy.linalg.lstsq`) and gets the same $m$ and $b$, so our two-formula class gives exactly its predictions.
- Together these answer the opening question: $m$ and $b$ come from setting both slopes of the total squared error to zero, which gives two short formulas.

## 8. Sources

### Built from


- CampusX, "Simple Linear Regression | Mathematical Formulation | Coding from Scratch", YouTube, https://www.youtube.com/watch?v=dXHIDLPKdmA
- Starmer, J. (StatQuest), "The Main Ideas of Fitting a Line to Data (The Main Ideas of Least Squares and Linear Regression)", YouTube, https://www.youtube.com/watch?v=PaFPbb66DxQ The intuition of turning a line and watching the squared errors change, behind Figure 6; we redraw it with our own data.

- Khan Academy, "Calculating the equation of a regression line", YouTube, https://www.youtube.com/watch?v=FGesqq22TCM (the slope as $r \times s_y/s_x$, section 5.1)

**Other references**

- scikit-learn API reference, `sklearn.linear_model.LinearRegression`, Notes section. scikit-learn.org.

## 9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Closed-form solution (G-398) | An answer given directly by a formula of ordinary operations. |
| Ordinary least squares (OLS, least squares) (G-1406) | The way linear regression finds its line with one formula (a closed-form method): the line with the smallest sum of squared errors. |
| Non-closed-form solution (G-1332) | An answer reached by improving a guess step by step. |
| Prediction ($\hat{y}$) (G-1550) | The value the model gives for an input whose true output is not known; the hat marks a prediction. |
| Loss function (error function) (G-706) | A formula that measures how wrong a model's predictions are; in deep learning, the error on one training row. |
| Partial derivative (G-1457) | The slope of a function of several variables in one variable, holding the others fixed. |
| Feature (G-772) | One piece of information about each example that a model uses (e.g. a student's CGPA). |
| Target (G-1949) | The output column we predict, such as the class; also called the label. |
| Observation (G-1374) | One record of a dataset: one row of the data table. |
| Best-fit line (G-280) | The line with the smallest total error over all the training points. |
| Linear regression (G-1094) | A model that predicts a number from the inputs by fitting the straight line (or, with several inputs, the plane) that runs closest to all the training points. |

| Term | Meaning |
|---|---|
| Derivative (G-595) | The slope of a function at a point: how fast the output changes as the input changes there. It is zero at the lowest point of a smooth curve, which is how training finds a minimum. |
| Gradient descent (G-862) | Finding the lowest point of a function by repeated small steps downhill. |
| Error (residual) (G-705) | The gap between an actual value and the model's prediction on one data point: actual minus predicted. |
| Residual sum of squares (sum of squared errors, SSE) (G-1684) | The squares of all the errors (residuals) added up; the quantity the best-fit line makes smallest, and a regression tree splits where the SSE of the two sides is smallest. |
| Mean squared error (MSE) (G-1201) | The average squared difference between actual and predicted values. |
| Tangent line (G-1945) | The line that touches a curve at one point with the curve's slope there; the limit of secant lines. |
| Chain rule (G-371) | To differentiate a function of a function, multiply the outer derivative by the inner derivative. |
| Mean (G-1203) | The average of the values; the centre of the data. |
| Covariance (G-496) | A number that says how two numerical features move together: positive if they rise together, negative if one rises as the other falls. Its size depends on the units. |
| Variance (of data) (G-2074) | The average squared distance of the values from their mean; the square of the standard deviation; divide by $n$ for a population (NumPy default) or $n - 1$ for a sample (pandas default). |
| Pearson correlation coefficient (Pearson's r) (G-1474) | A number from $-1$ to $+1$ that measures how closely two numerical columns follow a straight-line relationship ($+1$: they rise together exactly; 0: no straight-line link), written $r$; it is the one `df.corr()` computes. |
| Multiple linear regression (G-1279) | Linear regression with several input columns. |

| | |
|---|---|
| SGDRegressor (G-1783) | A scikit-learn model that does linear regression with stochastic gradient descent, step by step, so it can also learn from data arriving in small pieces. |